四位全加器实验

王珺桥 计42 2024010694

四位全加器

工作原理

基础的四位全加器是一个四位串行加法器，由四个一位全加器组成

$$F_n = A_n \oplus B_n \oplus C_{in}$$

$$C_{out} = A_n \cdot B_n + C_{in} \cdot (A_n \oplus B_n)$$

然后将上一个加法器的进位输出当做下一个加法器的进位输入即可

一位全加器

代码块

```
module full_adder(A, B , Cin, F, Cout);
    input wire A, B, Cin;
    output wire F, Cout;

    assign F = A ^ B ^ Cin;
    assign Cout = (A & B) | (Cin & (A ^ B));
endmodule
```

四位全加器

代码块

```
module adder (
    input wire[3:0] A, // 输入A
    input wire[3:0] B, //输入B
    input wire Cin, //输入的进位
    output wire[3:0] F, //输出
    output wire Cout //输出的进位
    );

    wire[2:0] C; //内部信号：用于存储各位之间的进位信号

    // 四个加法器，每个加法器处理一个位的加法运算，接收两个输入和一个进位
    full_adder adder0 (
```

```
        .A(A[0]),
        .B(B[0]),
        .Cin(Cin), //第一个加法器接收的进位是Cin
        .F(F[0]),
        .Cout(C[0]) // 中间的加法器输出的进位是内部进位
    );

    full_adder adder1 (
        .A(A[1]),
        .B(B[1]),
        .Cin(C[0]),
        .F(F[1]),
        .Cout(C[1])
    );

    full_adder adder2 (
        .A(A[2]),
        .B(B[2]),
        .Cin(C[1]),
        .F(F[2]),
        .Cout(C[2])
    );

    full_adder adder3 (
        .A(A[3]),
        .B(B[3]),
        .Cin(C[2]),
        .F(F[3]),
        .Cout(Cout) // 最后一个加法器输出的进位是Cout
    );

endmodule
```

管脚绑定

代码块

```
# A input
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports A[3]];
#IO1接插孔
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports A[2]];
#IO2接插孔
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports A[1]];
#IO3接插孔
set_property -dict {PACKAGE_PIN P21 IOSTANDARD LVCMOS33} [get_ports A[0]];
#IO4接插孔
```

```
6
7   # B input
8   set_property -dict {PACKAGE_PIN T21 IOSTANDARD LVCMOS33} [get_ports B[3]];
    #IO6接插孔
9   set_property -dict {PACKAGE_PIN U21 IOSTANDARD LVCMOS33} [get_ports B[2]];
    #IO7接插孔
10  set_property -dict {PACKAGE_PIN R21 IOSTANDARD LVCMOS33} [get_ports B[1]];
    #IO8接插孔
11  set_property -dict {PACKAGE_PIN R22 IOSTANDARD LVCMOS33} [get_ports B[0]];
    #IO9接插孔
12
13  # Cin input
14  set_property -dict {PACKAGE_PIN P22 IOSTANDARD LVCMOS33} [get_ports Cin];
    #IO5接插孔
15
16  # F output
17  set_property -dict {PACKAGE_PIN AB18 IOSTANDARD LVCMOS33} [get_ports F[3]];
     #IO17接插孔
18  set_property -dict {PACKAGE_PIN AA20 IOSTANDARD LVCMOS33} [get_ports F[2]];
     #IO18接插孔
19  set_property -dict {PACKAGE_PIN AB21 IOSTANDARD LVCMOS33} [get_ports F[1]];
     #IO19接插孔
20  set_property -dict {PACKAGE_PIN AA21 IOSTANDARD LVCMOS33} [get_ports F[0]];
     #IO20接插孔
21
22  # Cout output
23  set_property -dict {PACKAGE_PIN AA18 IOSTANDARD LVCMOS33} [get_ports Cout];
     #IO16接插孔
24
25  set_property CFGBVS VCCO [current_design]
26  set_property CONFIG_VOLTAGE 3.3 [current_design]
```

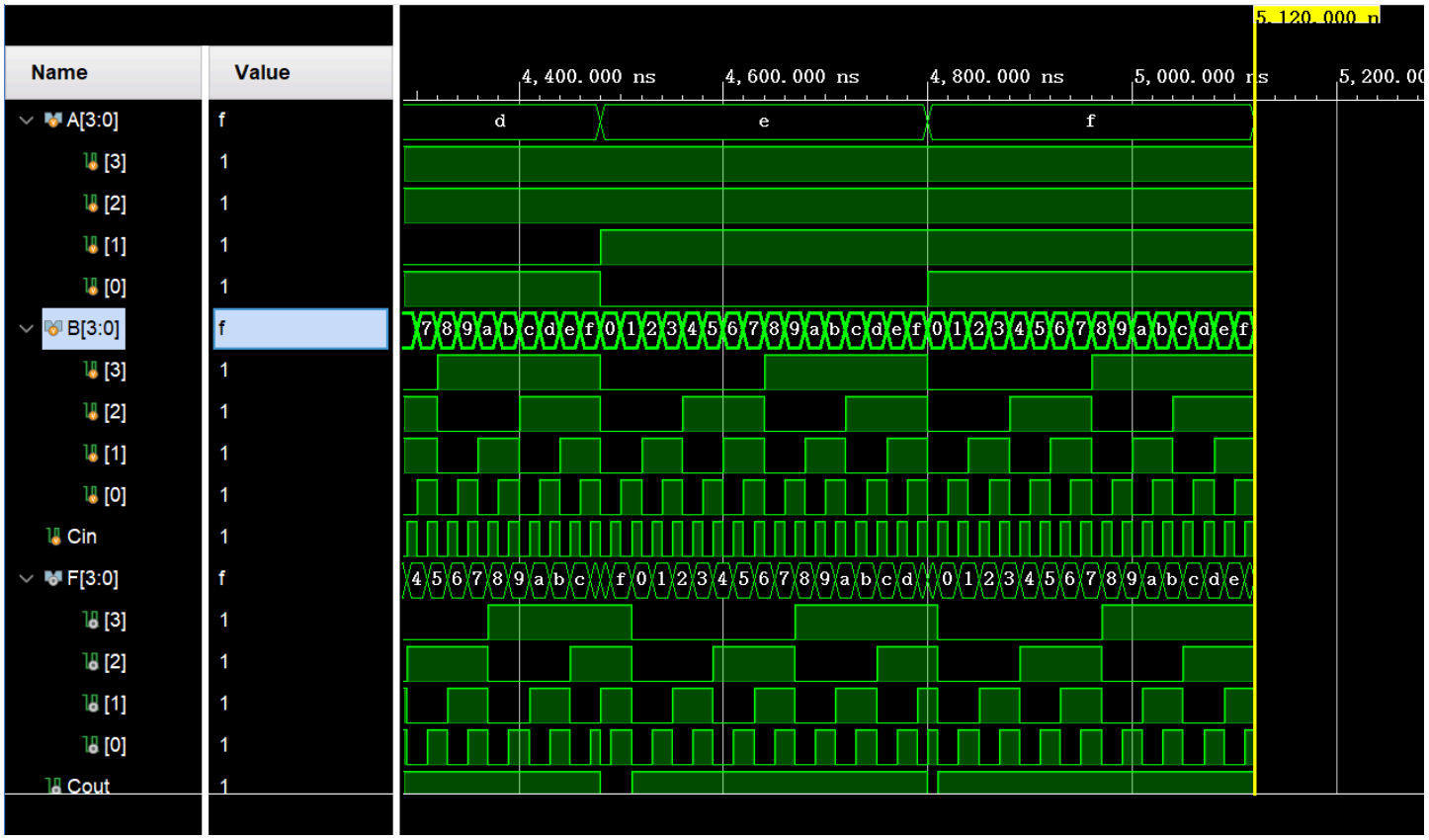
实验仿真

代码块

```
1   module tb;
2
3     reg  [3:0] A;
4     reg  [3:0] B;
5     reg        Cin;
6     wire [3:0] F;
7     wire       Cout;
8
9     integer i;
10
```

```
11      // DUT: Device Under Test, 被测试的设备
12      adder dut (
13        .A(A),
14        .B(B),
15        .Cin(Cin),
16        .F(F),
17        .Cout(Cout)
18      );
19
20      initial begin
21        // 遍历所有 A, B, Cin 组合，共 2^9 = 512 种
22        for (i = 0; i < 512; i = i + 1) begin
23          {A, B, Cin} = i;  //A 4位，B 4位，Cin 1位，共 9 位
24          #10;  // 每组输入间隔10ns
25          $display("A=%b, B=%b, Cin=%b | F=%b, Cout=%b", A, B, Cin, F, Cout);
26        end
27
28        $finish;
29      end
30
31  endmodule
```

仿真结果



如图，在最后一个状态时，A=1111，B=1111，Cin=1，F=1111，Cout=1，符合预期。

关于提高要求，在实验课上只实现了要求二和三即使用 SystemVerilog 自带的加法运算实现四位全加器和将结果采用十进制表示，并使用自带译码的七段数码管显示结果，结束后和同学讨论也尝试了超前进位，应该是没有问题，，

超前四位加法器

超前一位加法器

代码块

```
module pre_1_adder(A, B , Cin, F, Gi, Pi);
    input wire A, B, Cin;
    output wire F, Gi, Pi;
    assign Gi = A & B; // 进位生成 Gi：当 A 和 B 都为 1 时，Gi 为 1
    assign Pi = A | B; // 进位传播 Pi：当 A 或 B 为 1 时，Pi 为 1
    assign F = A ^ B ^ Cin;
endmodule
```

进位处理

代码块

```
module CLA_4(P,G,cin,Ci,Gm,Pm);
        input wire [3:0] P;  // 传递进位函数 Pi,Pi=Xi+Yi
        input wire [3:0] G;  // 产生进位函数 Gi,Gi=XiYi
        input wire cin;      // 输入：整体的进位输入 Cin
        output wire [4:1] Ci;// 输出：各位的进位信号 Ci[4:1]，即每一位的进位输出(Ci即向第i位的进位输出)
        output wire Gm;      // 输出：四位的进位生成信号 Gm
        output wire Pm;      // 输出：四位的进位传播信号 Pm
    // 数电第七次课PPT第25页，超前进位加法器
    assign Ci[1]=G[0]|P[0]&cin; // C1=G0+P0Cin
    assign Ci[2]=G[1]|P[1]&G[0]|P[1]&P[0]&cin;// C2 = G1 + P1G0 + P1P0Cin
    assign Ci[3]=G[2]|P[2]&G[1]|P[2]&P[1]&G[0]|P[2]&P[1]&P[0]&cin;
    assign Ci[4]=G[3]|P[3]&G[2]|P[3]&P[2]&G[1]|P[3]&P[2]&P[1]&G[0]|P[3]&P
    [2]&P[1]&P[0]&cin;
    assign Gm=G[3]|P[3]&G[2]|P[3]&P[2]&G[1]|P[3]&P[2]&P[1]&G[0]; //计算4位的产生进位函数，即C4
    assign Pm=P[3]&P[2]&P[1]&P[0];//4位的传递进位函数，4位全都是1才产生
endmodule
```

超前四位加法器

代码块

```
module pre_4_adder(
    input wire [3:0] A,
    input wire [3:0] B,
    input wire Cin,
    output wire [3:0] F,
    output Cout //进位输出
    );
    wire Pm; //每四位的进位输出
    wire Gm; //每四位的进位传送输出
    wire [4:1] CI; //每位的进位信号
    wire [3:0] Pi; //每位的进位函数
    wire [3:0] Gi; //每位的进位传送函数
pre_1_adder u0(
    .A(A[0]),
    .B(B[0]),
    .Cin(Cin),
    .F(F[0]),
    .Gi(Gi[0]),
    .Pi(Pi[0])
);
pre_1_adder u1(
    .A(A[1]),
    .B(B[1]),
    .Cin(CI[1]),
    .F(F[1]),
    .Gi(Gi[1]),
    .Pi(Pi[1])
);
pre_1_adder u2(
    .A(A[2]),
    .B(B[2]),
    .Cin(CI[2]),
    .F(F[2]),
    .Gi(Gi[2]),
    .Pi(Pi[2])
);
pre_1_adder u3(
    .A(A[3]),
    .B(B[3]),
    .Cin(CI[3]),
    .F(F[3]),
    .Gi(Gi[3]),
    .Pi(Pi[3])
);
CLA_4 uut
(
    .P(Pi),
```

```
48      .G(Gi),
49      .cin(Cin),
50      .Ci(CI),
51      .Gm(Gm),
52      .Pm(Pm)
53  );
54      assign Cout = CI[4];//最后一位的进位输出作为Cout
55  endmodule
```

sv自带加法实现的加法器

代码块

```
1   module self_add (
2       input [3:0] A,
3       input [3:0] B,
4       input Cin,
5       output [3:0] F,
6       output Cout
7   );
8       wire c1, c2, c3;  // 中间进位信号
9
10      // 逐位加法，每一位的进位都逐步传递
11      assign {c1, F[0]} = A[0] + B[0] + Cin;
12      assign {c2, F[1]} = A[1] + B[1] + c1;
13      assign {c3, F[2]} = A[2] + B[2] + c2;
14      assign {Cout, F[3]} = A[3] + B[3] + c3;
15
16  endmodule
```

F直接用加法实现，Cout通过逐位加法来实现

提高实验仿真

以自带加法实现的加法器为例进行仿真，仿真代码如下：

代码块

```
1   module tb_adder;
2
3     reg  [3:0] A;
4     reg  [3:0] B;
5     reg        Cin;
6     wire [3:0] F;
7     wire       Cout;
8
```

```
    integer i;

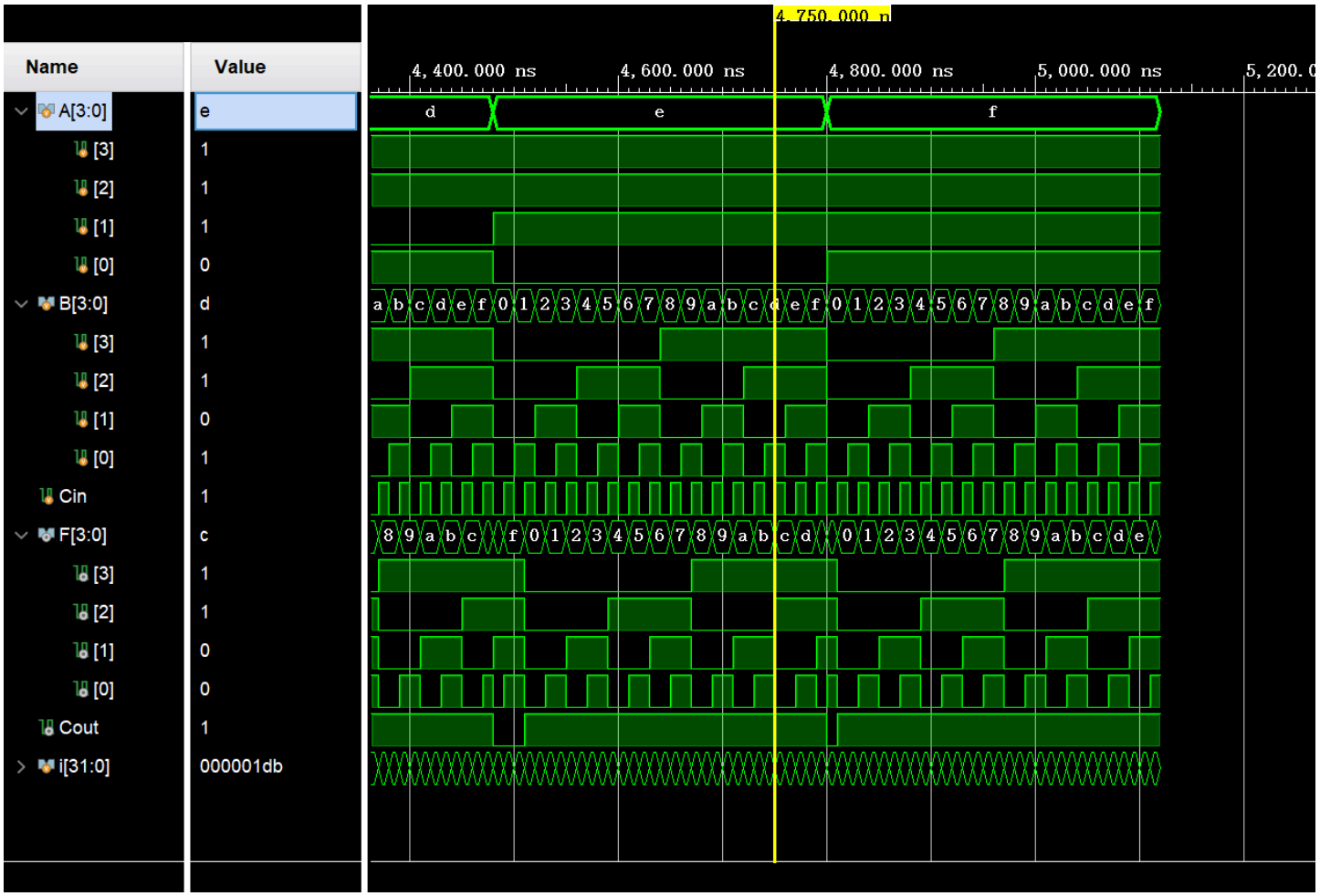
    // DUT: Device Under Test, 被测试的设备
    self_add dut (
      .A(A),
      .B(B),
      .Cin(Cin),
      .F(F),
      .Cout(Cout)
    );

    initial begin
      // 遍历所有 A, B, Cin 组合，共 2^9 = 512 种
      for (i = 0; i < 512; i = i + 1) begin
        {A, B, Cin} = i;  //A 4位，B 4位，Cin 1位，共 9 位
        #10;  // 每组输入间隔10ns
        $display("A=%b, B=%b, Cin=%b | F=%b, Cout=%b", A, B, Cin, F, Cout);
      end

      $finish;
    end

endmodule
```

仿真结果如下：



在图中情况下，A=1110，B=1101，Cin=1，F=1100，Cout=1，符合预期。

使用10进制来表示sv自带加法实现的加法器结果

代码实现

代码块

```
module topadd(
    input logic [3:0] A,
    input logic [3:0] B,
    input logic Cin,
    output logic [3:0] binF,
    output logic [3:0] decF,
    output logic Cout,
    output logic Cout1,
    output logic Cout2
);

    // 先进行二进制加法
    self_add self_adder (
        .A(A),
        .B(B),
        .Cin(Cin),
```

```
        .F(binF),
        .Cout(Cout)
    );

    // 使用十进制加法器处理二进制加法器的输出
    decimal_add dec_adder (
        .A(A),
        .B(B),
        .Cin(Cin),
        .F(decF),
        .Cout1(Cout1),
        .Cout2(Cout2)
    );

endmodule
```

管脚绑定

代码块

```
# A input
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports A[3]]; #IO1接插孔
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports A[2]]; #IO2接插孔
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports A[1]]; #IO3接插孔
set_property -dict {PACKAGE_PIN P21 IOSTANDARD LVCMOS33} [get_ports A[0]]; #IO4接插孔

# B input
set_property -dict {PACKAGE_PIN T21 IOSTANDARD LVCMOS33} [get_ports B[3]]; #IO6接插孔
set_property -dict {PACKAGE_PIN U21 IOSTANDARD LVCMOS33} [get_ports B[2]]; #IO7接插孔
set_property -dict {PACKAGE_PIN R21 IOSTANDARD LVCMOS33} [get_ports B[1]]; #IO8接插孔
set_property -dict {PACKAGE_PIN R22 IOSTANDARD LVCMOS33} [get_ports B[0]]; #IO9接插孔

# Cin input
set_property -dict {PACKAGE_PIN P22 IOSTANDARD LVCMOS33} [get_ports Cin]; #IO5接插孔

# decF output
```

```
17  set_property -dict {PACKAGE_PIN P20 IOSTANDARD LVCMOS33} [get_ports decF[0]];
        #IO10接插孔
18  set_property -dict {PACKAGE_PIN Y22 IOSTANDARD LVCMOS33} [get_ports decF[1]];
        #IO13接插孔
19  set_property -dict {PACKAGE_PIN Y21 IOSTANDARD LVCMOS33} [get_ports decF[2]];
       #IO14接插孔
20  set_property -dict {PACKAGE_PIN AB22 IOSTANDARD LVCMOS33} [get_ports decF[3]];
        #IO15接插孔
21
22  # binF output
23  set_property -dict {PACKAGE_PIN AB18 IOSTANDARD LVCMOS33} [get_ports binF[3]];
        #IO17接插孔
24  set_property -dict {PACKAGE_PIN AA20 IOSTANDARD LVCMOS33} [get_ports binF[2]];
        #IO18接插孔
25  set_property -dict {PACKAGE_PIN AB21 IOSTANDARD LVCMOS33} [get_ports binF[1]];
        #IO19接插孔
26  set_property -dict {PACKAGE_PIN AA21 IOSTANDARD LVCMOS33} [get_ports binF[0]];
        #IO20接插孔
27
28  # Cout output
29  set_property -dict {PACKAGE_PIN AA18 IOSTANDARD LVCMOS33} [get_ports Cout];
     #IO16接插孔
30  set_property -dict {PACKAGE_PIN W21 IOSTANDARD LVCMOS33} [get_ports Cout1];
    #IO11接 插 孔
31  set_property -dict {PACKAGE_PIN W22 IOSTANDARD LVCMOS33} [get_ports Cout2];
    #IO12接 插 孔
32
33  set_property CFGBVS VCCO [current_design]
34  set_property CONFIG_VOLTAGE 3.3 [current_design]
```

实验中遇到的问题和调试方法

1.忘记做超前进位，课下完成了

2.提高综合第一次验收时显示错误，检查时候发现把数码管位序连接错了，把引线反过来链接即可